Temco Platform

Modules · built Nov 10, 2026

Modules

A module is a real VS Code extension inside the application. Namespaces follow the module (../architecture/naming.md).

Register

Module	Purpose	Namespace	State
`temco-workbench`	the application's own surfaces: the Library, the dashboard, what needs doing	`temcoWorkbench`	**active**
`temco-ui`	where a page of ours opens, what a command argument means, and the shared page components (hero, figures, sections, cards)	`temcoUi`	**active**
`temco-store`	the platform's own memory: the catalog database, the selection, the naming rules	`temcoStore`	**active**
`temco-docs`	the registry, pages, checks, and the writes	`temcoDocs`	**active**
`temco-learn`	the published site: its app, the site folder, the panel, the preview, the downloads the deploy cannot make, and the push to `gh-pages`	`temcoLearn`	**active**
`temco-editor`	the editor for a page: the Markdown surface, its ribbon, its engine seam	`temcoEditor`	**active**
`temco-repositories`	the organisation, local copies, the working area, reading a source	`temcoRepositories`	**active**
`temco-ai`	the local runtime and models, the prompts, the panel, drafts and answers	`temcoAi`	**active**
`temco-projects`	projects, sources, outputs, builds	`temcoProjects`	**active**
`temco-sql`	registers, queries, export	`temcoSql`	reserved
`temco-nettools`	devices on the wire	`temcoNettools`	reserved
`temco-bacnet`	objects, points, trends	`temcoBacnet`	reserved
`temco-device-manager`	fleets, firmware, health	`temcoDevices`	reserved
`temco-tools`	the Tools container until those four modules exist: Devices, I/O Studio, BACnet Studio, Network and SQL Studio, as pages over a sample	`temcoTools`	**active**

States

State	Contents	Build effect
reserved	`README.md` only	the build ignores the folder: nothing is compiled, nothing ships, no container appears
active	`package.json`, `tsconfig.json`, `src/`	compiled and shipped as a built-in extension

A module becomes active on the day it has a first real feature. Reserving a module with a manifest would ship an extension with nothing in it, which is why a folder becomes a module only when it has something to do.

Ownership

`temco-store` — the platform's own memory, and only that: one SQLite file per window (sources read, projects, their sources, the pages a read found, builds, artifacts, commits, events), the **selection** (which project the window is on, and the page it last opened), and the naming rules the tables are keyed by (a slug). It has one command, no views and no settings. It is the base every other module stands on, and it is deliberately *not* `temco-sql`: that module is about the databases a **product** uses, and it neither owns nor knows about this file.

`temco-docs` — the content side: the registry of areas, chapters and pages, the structural checks, the **Workspace tree** (its own view), the pages, the Markdown surfaces, the reports and the builds, and the five writes. It owns the only write a page ever gets, and it publishes the documentation model other modules read (`src/api.ts`) — the Library and the dashboard are the workbench's, and they read that model rather than a copy of it.

`temco-ui` — the shared UI conventions, and nothing else: the editor column a page of ours opens in (one group, never a split), what a command argument names (a slug, a row, or a row's own command argument), and the page components a first screen is built from — the banded hero, the band of figures, a section heading, and a row of cards. It has no command, no view and no setting.

`temco-workbench` — the application's own surfaces, and the only module that is not a domain: the **Library** (the index of what every module has; each row's action is that module's own command), the **dashboard** (the application talking about itself), the **needs page**, which tab opens at startup, and which sidebar section comes forward when the project changes. It writes nothing and owns no data: its rows come from the store and from the documentation module's published model.

`temco-ai` — the local model: the runtime and models downloaded on request into the user's own folder, the prompts, the AI panel, drafts and their diffs, and the citations that open the page they name. It reads the documentation through `temco-docs` and never writes a page.

How two modules may talk

Only one shape is allowed, and it runs in one direction:

- **A provider publishes, a consumer declares.** `temco-ai` requires `temcocontrols.temco-docs` (`extensionDependencies`), so that module activates first and its exports — the sets, the pages, the selection, the editor column, one change signal — are there when the consumer's `activate()` runs. The shapes are spelled twice, once in each module (`src/api.ts` in the provider, `src/docs.ts` in the consumer), because an extension cannot import another extension's sources.
- **A consumer never requires a provider that requires it.** That is a cycle, and a cycle is a load-order bug waiting to happen. The second direction is always a *command* the first module calls and may find missing, and the caller gates what it shows on a context key it sets itself (`temcoDocs.aiInstalled`).

`temco-docs` and `temco-ai` are the worked example: one declared dependency, one optional command surface. `temco-store` is the other shape a module can have — a base with no surface of its own, which every other module requires and none calls back into.

`npm run check-temco` refuses the mistakes this section warns about: a dependency graph with a circle in it (VS Code disables *every* extension in a circle, so the application comes up with a silent sidebar), a menu, submenu or welcome link that names an id no module declares, a view contributed under a container no module declares (the workbench shows nothing at all — no icon, no panel, no error), a container id two modules both claim, and a module-level binding that collides with the CommonJS wrapper (`module`, `require`, …), which throws before the extension runs.

`temco-repositories` — the organisation and the local copies: the repository list, a source's files, cloning one branch with full history, `pull --ff-only`, removing the local copy, reading a file's text, **reading a source** — the registry a repository declares, from a folder or over the API, recorded into the store — and the **vocabulary of the repository's own documentation scripts** (which scripts exist and what each is for). It owns the working area (`temcoRepositories.workspaceFolder`, default `~/Temco`). It reads the platform's memory through `temco-store` and the shared page rules through `temco-ui`, and it needs nothing of `temco-docs`: what a registry *means* is that module's business, what a repository *has* is this one's.

`temco-projects` — a project as a first-class thing: name, version, audience, owner, the sources it is made of, the outputs it produces, and what building one left behind. Its pages are the list, the hub and the edit form. It reads sources and the repository's own scripts through `temco-repositories`, the platform's memory through `temco-store`, and it needs nothing from `temco-docs`: the pages a project *has* were recorded in the store by a read.

`temco-sql` — registers, queries and export against the databases the products use. It owns its own connections and storage.

`temco-nettools` — discovery, addressing, capture and inspection. The transport layer the device modules build on.

`temco-bacnet` — BACnet objects, points and trends, read through `temco-nettools` transports. It owns its object and point models and its storage.

`temco-device-manager` — fleets and health: inventory, firmware, status and the actions a person takes on a device. It reads devices through `temco-bacnet` and `temco-nettools`.

Folder

```text
extensions/temco-<module>/
├── README.md              purpose, identifiers, what it owns — the only file a reserved
├── package.json           id, engine range, commands, views, settings, activation
├── tsconfig.json          extends ../tsconfig.base.json
├── src/
│   ├── extension.ts       the only activation entry: register, wire, dispose
│   └── <feature>/
├── media/                 icons, webview css and js
├── prompts/               model instructions as text, where a module needs them
└── scripts/               module-local gates
```

Its documentation is `docs/temco-platform/modules/temco-<module>/` **in this repository**, not a `docs` folder inside the module. There is no `.vscodeignore`: the platform build packages a built-in module, `vsce` does not.

Contract

1. Modules talk through commands, settings, files and events. No module imports another module's source.
2. The catalog belongs to `temco-docs`. Other modules read it through commands.
3. Each module owns its own storage under its own extension id — never inside a repository, never committed.
4. One module, one namespace. A command id is registered exactly once.
5. A module is a built-in extension. It uses the published `vscode` API only and never the shell's internals, so it still runs in an ordinary VS Code window.
6. A module never assumes the shell.

Adding a module

1. Create the folder with the shape above.

2. Write `package.json` with its id, commands, views, settings and activation events.
3. `tsconfig.json` extends `../tsconfig.base.json`.
4. Store everything under its own id's storage.
5. Add its gate to the release list: `tsc`, plus its own behaviour check where it has one.
6. Write its page in `docs/temco-platform/modules/temco-<module>/`: commands, settings, storage, and what it must not own.
7. Compile it with the platform build (`npm run gulp compile-extension:temco-<module>`) and check it loads.

Nothing else is required: the build picks up any folder with a `package.json`, the root install brings in its dependencies, and web builds skip it automatically.